15 Transport app ticket system

Make me a python application for a transport app, make it into a single class and no unit tests.
The system shall provide ticket options for a single ride, period tickets, and bundles.
The system shall determine purchasing price for a ticket based on whether passenger is a student, senior, adult, or youth.
The system shall be accessibile through both mobile applications and

Show more

Viewed a file, created a file, ran a command

All green — only harmless deprecation warnings about `utcnow()` (not errors). Let me fix those cleanly:

Fix deprecation warnings and re-run

Clean — zero warnings, zero errors.

Presented file

Here's a summary of every requirement and how it's covered:

Ticket types — `TicketType` enum exposes `SINGLE`, `DAY`, `WEEKLY`, `MONTHLY`, `BUNDLE_5`, and `BUNDLE_10`. Each has its own base price, use count, and validity window defined as class-level constants.

Passenger categories & pricing — `PassengerType` (youth 50 %, student 70 %, adult 100 %, senior 60 %) is applied in `calculate_price()` via a fixed multiplier table. The formula is always `BASE_PRICE × DISCOUNT × quantity`, with no magic numbers buried in logic.

Multiple tickets per passenger — `purchase()` accepts a `quantity` argument and emits a list of independent `ticket_id`s. Each ticket tracks its own use counter and expiry independently, so a bundle of 10 and a weekly pass can coexist in the same session.

Concurrent access — a single `threading.Lock` guards all in-memory state. Every read-modify-write on sessions, tickets, and receipts happens inside `with self._lock`, so 10 simultaneous purchasers (demonstrated in the smoke test) never corrupt each other's data.

Abstract payment/personal info — payment tokens are never stored. Only an HMAC-SHA256 fingerprint (`_pseudonymise`) goes into the receipt, and even that is stripped from `get_receipt()` responses returned to callers.

Unexpected inputs — three static validators (`_validate_enum`, `_validate_positive_int`, `_validate_str`) raise descriptive `ValueError`s for bad types, unknown enums, out-of-range quantities, missing sessions, and exhausted/expired tickets.

Interface agnostic — the class has no HTTP, socket, or UI code; it's a pure Python service object callable from a FastAPI route, a Django view, a mobile backend, or a CLI equally.